

8. Insertion Sort with Duplicate Elements

Aim

To write a Python program to sort an array containing **duplicate elements** using the **Insertion Sort** algorithm and analyze its complexity.

Algorithm

1. Start.
2. Take an array containing duplicate elements.
3. Start from the second element.
4. Store the current element as `key`.
5. Compare `key` with the elements before it.
6. Shift elements greater than `key` one position to the right.
7. Insert `key` in its correct position.
8. Repeat until all elements are sorted.
9. Print the sorted array.
10. Stop.

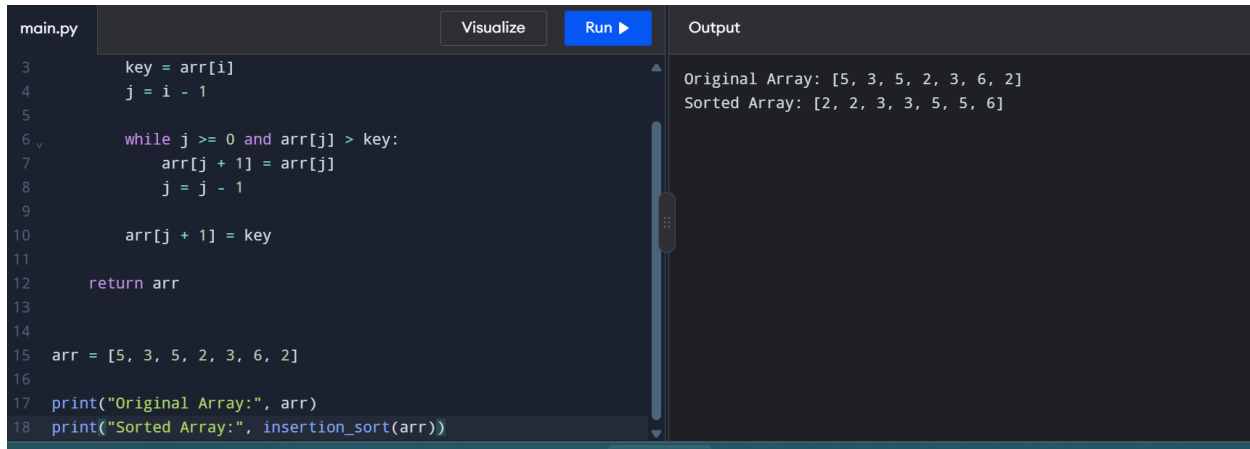
Source Code

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j = j - 1  
        arr[j + 1] = key  
    return arr  
  
arr = [5, 3, 5, 2, 3, 6, 2]
```

```
print("Original Array:", arr)

print("Sorted Array:", insertion_sort(arr))
```

Output

A screenshot of a code editor interface. On the left, a file named 'main.py' contains Python code for an insertion sort algorithm. The code defines a function 'insertion_sort' that takes an array 'arr' and returns a sorted array. It uses a while loop to shift elements greater than the 'key' to the right and then inserts the 'key' at the correct position. The main part of the code initializes an array 'arr' with the values [5, 3, 5, 2, 3, 6, 2] and prints both the original and sorted arrays. On the right, the 'Output' panel shows the results: 'Original Array: [5, 3, 5, 2, 3, 6, 2]' and 'Sorted Array: [2, 2, 3, 3, 5, 5, 6]'.

```
main.py Visualize Run ▶ Output
3     key = arr[i]
4     j = i - 1
5
6     while j >= 0 and arr[j] > key:
7         arr[j + 1] = arr[j]
8         j = j - 1
9
10    arr[j + 1] = key
11
12    return arr
13
14
15    arr = [5, 3, 5, 2, 3, 6, 2]
16
17    print("Original Array:", arr)
18    print("Sorted Array:", insertion_sort(arr))

Output
Original Array: [5, 3, 5, 2, 3, 6, 2]
Sorted Array: [2, 2, 3, 3, 5, 5, 6]
```

Complexity Analysis

- **Best Case:** $O(n)$ — array is already sorted.
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$ — array is in reverse order.
- **Space Complexity:** $O(1)$

Note: Duplicate elements are handled normally. The $>$ condition also makes Insertion Sort **stable**, meaning equal elements keep their original relative order.

Result

Thus, the **Insertion Sort** algorithm was successfully implemented on an array containing duplicate elements. The worst-case time complexity is **$O(n^2)$** and the space complexity is **$O(1)$** .